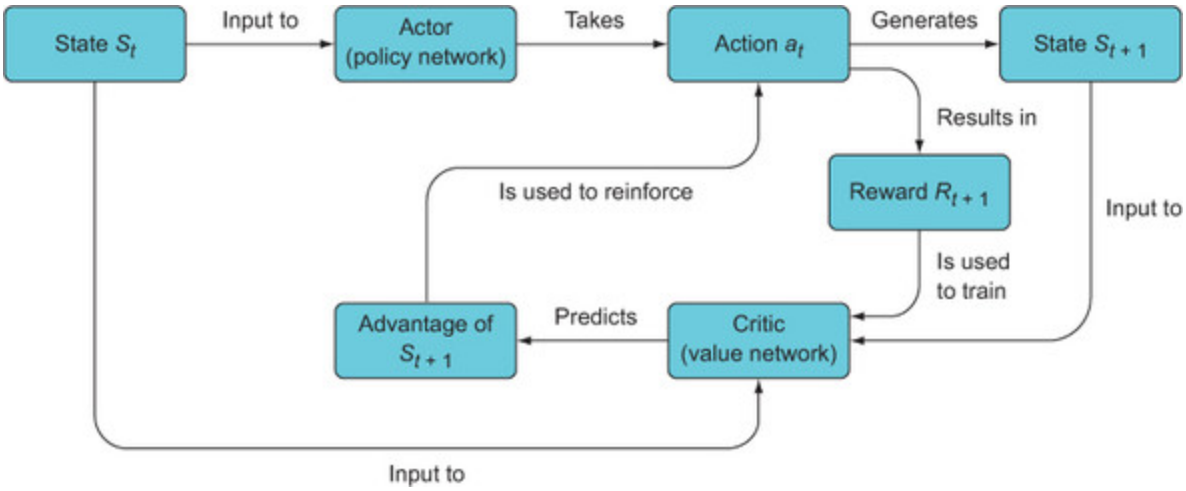
coming from the environment, but the sequence of rewards will depend on the actions taken by the actor, so the actor affects the critic too, albeit more indirectly (figure 5.6).

Figure 5.6. The general overview of actor-critic models. First, the actor predicts the best action and chooses the action to take, which generates a new state. The critic network computes the value of the old state and the new state. The relative value of S_{t+1} is called its advantage, and this is the signal used to reinforce the action that was taken by the actor.



The actor is trained in part by using signals coming from the critic, but how exactly do we train a state-value function as opposed to the action value (Q) functions we're more accustomed to? With action values, we computed the expected return (the sum of future discounted rewards) for a given state-action pair. Hence, we could predict whether a state-action pair would result in a nice positive reward, a bad negative reward, or something in between. But recall that with our DQN, our Q-network returned separate action values for each possible discrete action, so if we employ a reasonable policy like epsilon-greedy, the state value will essentially be the highest action value. Thus, the state-value function just computes this highest action value rather than separately computing action values for each action.

5.2. Distributed training

As we mentioned in the introduction, our goal in this chapter is to implement a model called distributed advantage actor-critic (DA2C), and